

Independent Study Project Brief

Faculty of Information Technologies & Business Systems

Program: Business Systems (Year 1 — Introductory Track) **Project Code:** BS-FINDB-2026
Student Name: _____

Student ID: _____

Academic Supervisor: Prof. Asen Grozdanov

Project Duration: 5 Weeks (May 1 – June 5, 2026)

Consultation Schedule: 4 sessions × 1 hour each (see Section 6)

1. Project Title

"Designing and Analyzing a Financial Database: Currency and Market Data for the Kazakhstan Context"

2. Background and Motivation

One of the most practical skills a business systems graduate can develop early on is the ability to work with structured data — specifically, to build the containers that hold that data (databases), understand how they are organized, and then extract meaningful insight from them. This becomes especially relevant when the data in question is financial in nature, because financial data is everywhere: banks use it, regulators publish it, and every business that deals in more than one currency encounters it daily.

Kazakhstan presents an interesting environment for this kind of project. The National Bank of Kazakhstan (NBK) publishes official exchange rate data for the tenge (KZT) against major world currencies on a daily basis and makes it freely accessible. The Kazakhstan Stock Exchange (KASE) also maintains public market data. Both sources are reliable, locally relevant, and require no paid subscription or VPN to access — which makes them well-suited to a first practical project of this kind.

Over five weeks, the student will move through the full cycle: understanding what a relational database is and why its structure matters, designing a small schema that fits a real financial use

case, populating it with real data from NBK, and finally running queries and simple analyses that answer practical business questions. The project is intentionally scoped to be achievable within the timeframe and with tools that are freely available and run locally — no cloud accounts are required.

3. Learning Objectives

By the end of the project, the student should be able to:

1. Explain the core concepts of relational databases — tables, primary keys, foreign keys, normalization — without needing to look them up.
 2. Draw and justify a simple Entity-Relationship (ER) diagram for a defined business problem.
 3. Write SQL queries of moderate complexity (SELECT with JOINS, GROUP BY, aggregate functions, date filtering).
 4. Retrieve structured data from a public financial source (NBK API or published datasets) and load it into a local database.
 5. Perform basic financial data analysis — identifying trends, comparing periods, calculating averages and volatility — using SQL and, optionally, Python with pandas.
 6. Produce a written technical report that documents design decisions clearly enough for another student to reproduce the work.
-

4. Project Description

4.1 The Core Idea

The student will build a local relational database that stores historical KZT exchange rate data for at least five major currencies (USD, EUR, RUB, CNY, GBP), sourced from the NBK. On top of this database, the student will run analytical queries that answer questions such as:

- What was the average USD/KZT rate in April vs. March of this year?
- Which month in the last 12 months showed the greatest tenge depreciation against the dollar?
- How correlated are the EUR/KZT and USD/KZT movements over a given period?

These are not abstract exercises — they are exactly the kinds of questions a junior analyst at a Kazakhstani bank or trading company might be asked to answer on their first week.

4.2 Scope Boundaries

To keep the project realistic for a first-year student with 5 weeks, the scope is deliberately limited:

In scope:

- Design of a relational database schema (2–4 tables)
- Data sourcing from NBK open data (exchange rates, possibly NBK base rate history)
- Data loading via CSV import or a simple Python script
- SQL query writing and analysis
- A short written report and an ER diagram

Out of scope (for this project):

- Web applications or dashboards
- Real-time data streaming
- Machine learning or forecasting models
- Connecting to KASE trading systems (requires registration)

These topics are good for follow-up projects, but trying to include them now would stretch the timeline beyond what is reasonable.

4.3 Technology Stack

All tools listed here are free, open-source, and run on Windows or Linux without admin-level cloud accounts. They are all available for download without restrictions in Kazakhstan.

Tool	Purpose	Where to Get It
PostgreSQL 16	Main database engine	postgresql.org
DBeaver Community	Database GUI (query editor, ER diagrams)	dbeaver.io
Python 3.11+	Data loading scripts, optional analysis	python.org
pandas	Data manipulation library	via pip
Jupyter Notebook	Optional — for presenting analysis	via pip
VS Code	Text editor / script editor	code.visualstudio.com
Git	Version control for project files	git-scm.com

Note on internet access: All data retrieval will use the NBK's published API endpoint (https://nationalbank.kz/rss/get_rates.cfm) or their downloadable Excel/CSV archives, both of which are accessible without a VPN. If any tool download is slow, the student should reach out before Session 2 so an alternative can be arranged.

5. Weekly Work Plan

The five weeks are structured so that each builds directly on the last. The first two weeks are heavier on learning and design; the middle week is the main build; the last two weeks are analysis and write-up. The student should keep a short log (even just a few bullet points per day) of what they worked on — this will be useful for the report and for the consultation sessions.

Week 1 — Foundations and Setup

Dates: May 1–7, 2026 **Consultation Session 1: May 1, 2026 @ 8:00 PM Almaty Time**

The first session happens right at the start of Week 1, which is intentional. It gives the student a chance to ask clarifying questions before doing any significant work, and to confirm the setup is correct.

Tasks:

- Install and configure PostgreSQL and DBeaver. Create a test database and run a few basic queries (`CREATE TABLE`, `INSERT`, `SELECT`) just to confirm everything works.
- Read the following (free, online): chapters 1–3 of "Learning SQL" by Alan Beaulieu (available via Open Library at openlibrary.org) — covers the relational model, basic SQL syntax, and filtering.
- Browse the NBK website (nationalbank.kz) and explore what data is available. Download a sample exchange rate file manually to understand its structure.
- Draft a first version of the ER diagram — even a rough sketch on paper is fine for now. The question to answer: what entities does this system need, and how are they related?

Deliverable for end of week: A working PostgreSQL installation, a hand-drawn or DBeaver-drawn ER diagram (first draft), and a one-page note explaining the data source and what fields it contains.

Week 2 — Database Design and Data Modeling

Dates: May 8–14, 2026 **Consultation Session 2: May 8, 2026 @ 8:00 PM Almaty Time**

Tasks:

- Finalize the database schema. The recommended structure is:

- `currencies` — stores each currency code, full name, and country (e.g., USD, United States Dollar, US)
- `exchange_rates` — stores one row per currency per date, with the KZT rate value and the quantity multiplier (NBK sometimes quotes rates per 100 units)
- `nbk_base_rate` — optionally, a table for the NBK base interest rate over time, which adds context to currency movements

This is not the only valid design, and the student is encouraged to justify their choices rather than copy this structure blindly.

- Learn about normalization. The goal is to understand why we split data into separate tables rather than putting everything in one large spreadsheet. First Normal Form (1NF), Second Normal Form (2NF), and Third Normal Form (3NF) should be reviewed — a good short explanation is available on vertabelo.com/blog/.
- Write the `CREATE TABLE` statements in SQL and execute them in DBeaver. Add appropriate constraints: primary keys, foreign keys, NOT NULL where it makes sense, and a UNIQUE constraint on the (`currency_id`, `rate_date`) combination in the `exchange_rates` table.

Deliverable for end of week: Final ER diagram (can be exported from DBeaver), the SQL `CREATE TABLE` script saved as a `.sql` file, and a short written explanation (half a page) of the normalization choices made.

Week 3 — Data Collection and Loading

Dates: May 15–21, 2026 **Consultation Session 3: May 15, 2026 @ 8:00 PM Almaty Time**

This is the most technical week of the project. The student will retrieve real NBK exchange rate data and get it into the database in a clean, usable form.

Tasks:

- Write a Python script to fetch exchange rate data from the NBK API. The endpoint `https://nationalbank.kz/rss/get_rates.cfm?fdate=DD.MM.YYYY` returns XML for a given date. The script should:
 - Accept a date range as input
 - Parse the XML response
 - Write the results to a CSV file

A working example of the request (in Python using the `requests` and `xml.etree.ElementTree` libraries):

```
import requests
import xml.etree.ElementTree as ET
```

```

date_str = "01.04.2026"
url = f"https://nationalbank.kz/rss/get_rates.cfm?fdate={date_str}"
response = requests.get(url)
root = ET.fromstring(response.content)

for item in root.findall("./item"):
    title = item.find("title").text
    description = item.find("description").text
    quantity = item.find("quant").text
    print(title, description, quantity)

```

- Collect data for at least the last 12 months (May 2025 – April 2026). This will produce roughly 250–300 rows per currency for 5 currencies — a manageable dataset.
- Load the CSV into PostgreSQL. This can be done with DBeaver's import wizard or with the `\COPY` command in `psql`. Validate the load by running a count query and checking a few known values against the NBK website.
- Handle data quality issues: some dates are weekends or holidays when the NBK does not publish rates. The script or loading process should handle missing dates gracefully (skip or fill with the previous business day's rate — document whichever choice is made).

Deliverable for end of week: The Python data collection script (saved as `.py`), the loaded database with at least 12 months of exchange rate data, and a brief note on any data quality issues encountered.

Week 4 — SQL Analysis and Querying

Dates: May 22–28, 2026

No consultation session this week — use the time to focus on the analysis.

Tasks:

The student will write SQL queries to answer at least five of the following questions. These should be written in DBeaver, tested, and saved as a `.sql` file:

1. What was the average USD/KZT exchange rate for each month in the last 12 months?
2. Which date had the highest USD/KZT rate? Which had the lowest?
3. Calculate the month-over-month percentage change in the EUR/KZT rate.
4. How many days in the last year did the KZT/USD rate move by more than 1% compared to the previous business day?
5. Which of the five currencies showed the greatest overall depreciation of the tenge over the full 12-month period?
6. What is the standard deviation of the daily USD/KZT rate for each quarter? (This is a rough proxy for volatility.)
7. Compare the average RUB/KZT rate in the first half vs. second half of 2025.

For students who want to go further: export the query results to a CSV and load them into a Jupyter Notebook. Use `matplotlib` or just `pandas` to create a simple line chart of USD/KZT over the year. This is optional but demonstrates good initiative.

Deliverable for end of week: A `.sql` file containing all queries with brief comments explaining what each one does, and a short paragraph (for each query) interpreting the result in plain language — not just "the output was X" but "this means that..."

Week 5 — Report Writing and Wrap-Up

Dates: May 29 – June 5, 2026 **Consultation Session 4: May 29, 2026 @ 8:00 PM Almaty Time**

The final session is a review of the draft report and a discussion of findings. The student should come with a near-complete draft — not a finished product, but something substantial enough to give useful feedback on.

Tasks:

- Write the project report (see Section 7 for structure and requirements).
 - Review and clean up all project files — scripts, SQL files, diagrams — so they are organized and someone else could understand them.
 - Prepare a short verbal summary (5 minutes) of the most interesting finding from the analysis. What did the data actually tell us about the tenge's behavior over the past year? This is good practice for professional communication.
 - Finalize and submit all deliverables by June 5, 2026.
-

6. Consultation Sessions — Schedule Summary

Session	Date	Time	Main Focus
Session 1	May 1, 2026 (Friday)	8:00 PM Almaty Time	Project kickoff, setup check, clarify scope
Session 2	May 8, 2026 (Friday)	8:00 PM Almaty Time	Schema review, normalization discussion
Session 3	May 15, 2026 (Friday)	8:00 PM Almaty Time	Data loading issues, query planning
Session 4	May 29, 2026 (Friday)	8:00 PM Almaty Time	Report draft review, findings discussion

Sessions will be held online. Students should come with specific questions prepared — the hour goes quickly. If a question can be answered asynchronously (e.g., by email or message), that is preferred so session time can be used for things that genuinely require back-and-forth.

7. Final Deliverables and Report Structure

7.1 Files to Submit

All files should be organized in a single folder, zipped, and submitted via the program portal by **June 5, 2026, 11:59 PM Almaty Time.**

```
/BS_FINDB_2026_[StudentID]/
  /sql/
    01_schema_create.sql
    02_queries_analysis.sql
  /scripts/
    fetch_nbk_rates.py
  /data/
    exchange_rates_raw.csv      ← the raw data before loading
  /diagrams/
    er_diagram_final.png
  report.pdf                   ← or .docx
  README.txt                   ← brief guide to what each file is
```

7.2 Report Structure

The report should be 2,000–3,500 words. It is a technical document, not an essay — clear structure and precise language matter more than a high word count.

Recommended sections:

1. **Introduction** — What is the project about and why does it matter? (Half a page. Be specific — avoid vague statements like "databases are important in modern business.")
2. **Data Source and Collection** — Where does the data come from? What does the NBK API return, and how was it retrieved? What data quality issues were found, and how were they handled?
3. **Database Design** — Present and explain the ER diagram. Justify the choice of tables and the relationships between them. Explain normalization decisions — what was avoided and why.
4. **SQL Analysis** — For each query: state the business question, show the SQL (formatted clearly), show a sample of the output (a small table is fine), and interpret the result. This section will likely be the longest.
5. **Key Findings** — What did the analysis actually reveal about KZT exchange rate behavior in the past 12 months? This is where the student steps back from the technical detail and says something meaningful about the data itself.

6. **Reflections and Next Steps** — What would you do differently if you had another two weeks? What questions did the project raise that it did not answer? What would the next version of this database include?
7. **References** — NBK data source, any textbooks or tutorials consulted, PostgreSQL/Python documentation.

A note on writing: The report should be written in the student's own voice. It does not need to use complex vocabulary or imitate academic journal papers. What it does need is precision — say exactly what was done, not approximately. If a decision was made for a pragmatic reason ("this approach was simpler and the difference in results was negligible"), say so. That kind of honesty is more valuable than pretending every decision was theoretically optimal.

8. Assessment Criteria

Criterion	Weight	What is Being Evaluated
Database Design	25%	Is the schema logical? Are normalization principles applied correctly? Are constraints appropriate?
Data Collection & Loading	20%	Is the Python script functional and readable? Is the data complete and correctly loaded?
SQL Query Quality	25%	Are queries correct? Do they answer the stated question? Is SQL style clear and readable?
Analysis and Interpretation	20%	Are findings interpreted meaningfully, not just restated as numbers?
Report Clarity	10%	Is the report well-organized, precise, and complete?

9. Academic Integrity

All code and SQL submitted must be the student's own work. The student is permitted — and encouraged — to use documentation, tutorials, and reference materials. If a snippet of code is adapted from a tutorial or Stack Overflow answer, it should be noted with a comment in the code (e.g., `# adapted from postgresql.org/docs`). Submitting code that the student cannot explain verbally is a problem regardless of its source.

This project is structured so that sessions can include a brief verbal check-in on specific parts of the work. The student should be able to walk through their schema design or their Python script and explain their choices.

10. Resources

Primary Data Source:

- National Bank of Kazakhstan — exchange rates:
<https://nationalbank.kz/en/rates/ratesdynamic>
- NBK API (XML):
https://nationalbank.kz/rss/get_rates.cfm?fdate=DD.MM.YYYY

Learning Materials (all free):

- "Learning SQL" — Alan Beaulieu (Open Library: openlibrary.org)
- PostgreSQL official documentation: postgresql.org/docs/current/
- "SQL for Data Analysis" — Cathy Tanimura (O'Reilly; check if your university library has access)
- Vertabelo Academy blog on normalization: vertabelo.com/blog/
- pandas documentation: pandas.pydata.org/docs/
- W3Schools SQL reference (good for quick lookups): w3schools.com/sql/

Tooling (all free, no account required to download):

- PostgreSQL: postgresql.org/download/
- DBeaver Community Edition: dbeaver.io/download/
- Python: python.org/downloads/
- VS Code: code.visualstudio.com

This project brief was prepared for the Business Systems introductory track. Questions about the project scope or deliverables should be directed to the academic supervisor at least 48 hours before a scheduled consultation session to allow adequate preparation time.

Document version: 1.0 | Issued: April 2026